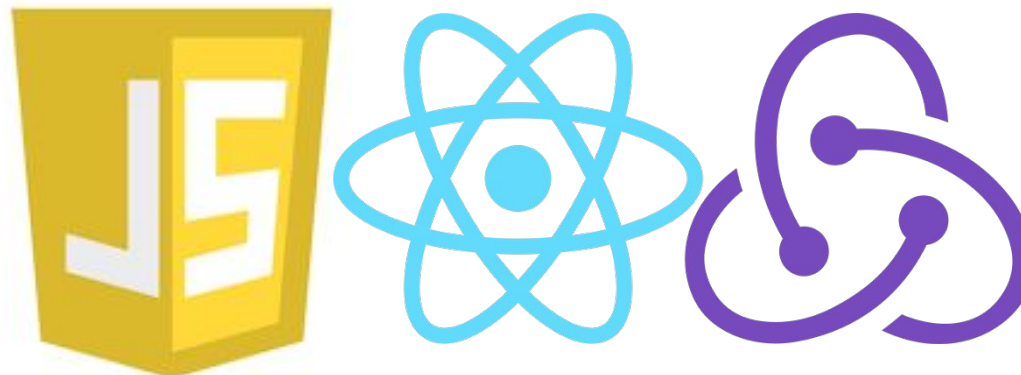


Лекція 13

Сучасний фронтенд JavaScript. React. Redux.

(Частина 2)



1. React.

Повний посібник та документацію по React можна знайти за посиланням:

<https://17.reactjs.org/docs/getting-started.html>

React — це декларативна, ефективна та гнучка JavaScript **бібліотека** для створення інтерфейсів користувача. Вона дозволяє вам збирати складний UI з маленьких ізольованих шматочків коду, які називаються «**компонентами**».

1.1. React. Component.

Приклад найпростішого компонента React:

```
import React from "react";
class App extends React.Component {
  render() {
    return (
      <div>
        We are starting a loooong, but interesting way...
      </div>
    );
  }
}
export default App;
```

render() - функція, яка повертає код, який відповідає за відображення DOM-дерева

1.2. React. Props.

Компонент приймає параметри, які називаються **props** (скорочення від properties – властивості)

Виклик компонента, з передачею в нього **props**:

```
<App  
  age={20}  
  name="Пацан"  
>
```

Функція **render()** компоненти, що викликається:

```
render() {  
  console.log(this.props); // {name: "Пацан", age: 20}  
  return (  
    <div>  
      <div>  
        {'Hello ' + this.props.name + ', this is how to show your name.'}  
      </div>  
      <div>  
        {'Ah, ${this.props.name}, we also can concat strings this way!`'  
      </div>  
    </div>  
  );  
}
```

1.3. React. State і constructor.

Компоненти React можуть отримати стан, встановлюючи **this.state** у **конструкторі**. **this.state** повинно розглядатися як приватна властивість React-компоненти, що визначається ним самим так само як і функція **this.setState()**, що оновлює **this.state**, і ініціює повторний виклик функції **render()**, щоб відобразити зміни

```
class App extends React.Component {
  constructor(props){
    super(props);
    this.state = {
      clickedCount: 0,
    };
  }
  render() {
    console.log(this.state); //До натискання: {clickedCount: 0}, після: {clickedCount: 1}
    return (
      <button
        onClick={() => this.setState({clickedCount: this.state.clickedCount + 1})}
      >
        Ну натисніть... Ну будь лааааска...
      </button>
    );
  }
}
```

1.4. React. Життєвий цикл компоненти.

Кожна компонента має **життєвий цикл** - набір вбудованих функцій, які послідовно викликаються за певних умов. Далі наведено список **лише основних** функцій життєвого циклу, у послідовності їх викликів:

- **constructor(props)** - викликається під час створення компоненти, і приймає параметрах props. Першим рядком конструктора необхідно викликати конструктор предка: **super(props)**. У конструкторі ініціалізується **this.state** компоненти, в який можна перекласти значення з **props** за необхідності.
- **render()** - викликається після конструктора і після оновлення хоча б одного поля з **this.props** або з **this.state**. Необхідний для відображення HTML.
- **componentDidMount()** - викликається після того, як компонент відобразився на сторінці. У цій функції потрібно викликати функції, які роблять запити отримання даних з back-end.
- **componentDidUpdate(prevProps, prevState)** - викликається після **render()**, після того, як компонент оновився на сторінці (після оновлення хоча б одного поля з **this.props** або **this.state**). Тут можна робити перевірку, якщо **this.props.someRequestParam !== prevProps.someRequestParam**, то знову надіслати запит на отримання даних з back-end.
- **componentWillUnmount()** - викликається перед тим, як компонент буде видалено зі сторінки. Тут можна очищати ресурси (**this.state** чи **редьюсерів**, про які ми поговоримо пізніше).

2. Redux.

Redux є передбачуваним контейнером стану JavaScript додатків. Redux пропонує думати про додаток, як про початковий стан, що модифікується послідовністю дій (actions).

Підключення до проекту:

```
npm install redux
```

```
npm install react-redux
```

2.1. Redux. Store.

У Redux весь стан додатку зберігається у вигляді єдиного об'єкта.

Такий об'єкт називається **state**, а сутність, яка ним керує - **store**.

Під **станом додатку** розуміється набір всіх даних, які потрібно відобразити на UI. Це дані, які будуть завантажені з BE (back-end).

Також туди можуть потрапляти ті дані, які вносить користувач взаємодіючи з елементами GUI, перед збереженням або просто відправкою на BE.

2.2. Redux. Store. Завдання, що вирішуються.

Store вирішує такі завдання:

- містить стан додатку (application state);
- надає доступ до стану за допомогою **getState()**;
- надає можливість оновлення стану за допомогою **dispatch(action)**;

2.3. Redux. Store. Приклад.

Приклад стану додатку, яка зберігає store:

```
{  
  students: {  
    isLoading: false,  
    isErrorLoad: false,  
    list: [  
      { id: 1, name: 'Petro' },  
      { id: 2, name: 'Stepan' },  
    ],  
  },  
  tickets: {  
    isLoading: true,  
    isErrorLoad: false,  
    list: [],  
  },  
}
```

2.4. Redux. Store. Внесення змін.

Єдиний спосіб **змінити стан** – це застосувати до store **action** – об'єкт, який описує, що сталося.

Змінити стан — це означає **створити новий об'єкт стану**, який вже міститиме потрібні зміни, а старий об'єкт стану буде забутий.

2.5. Redux. Action.

Як було сказано раніше: єдиний спосіб змінити стан - це **action**.

Що таке **action**? І що означає **застосувати**?

Action — це об'єкт з обов'язковим полем **type**. Саме по цьому полю, store визначає який код необхідно виконати (який **reducer** викликати), щоб змінити state.

Приклад 1:

```
{ type: 'REQUEST_STUDENTS' }
```

Приклад 2:

```
{  
  type: 'RECEIVE_STUDENTS',  
  students: [  
    { id: 1, name: 'Petro' },  
    { id: 2, name: 'Stepan' }  
  ],  
}
```

2.6. Redux. Action. Застосування.

Так що означає **застосовувати** action?

Застосовувати action — це означає викликати у store функцію `dispatch()`, і передати в нього action.

Генератори екшенів (Action Creators) — це функції, що створюють екшени. У Redux генератори екшенів просто повертають action:

// Генератор екшена

```
const receiveStudents = (students) => ({  
  type: 'RECEIVE_STUDENTS',  
  students,  
});
```

// Застосування екшена

```
store.dispatch(receiveStudents(students));
```

2.7. Redux. Reducers.

Отже, ми вже знаємо, що для того, щоб змінити стан (state) нашого store, нам потрібно застосувати до цього store якийсь action, який повідомляє що ж змінилося в нашій системі.

Редюсер (reducer) — це чиста функція, яка приймає попередній стан та екшен (state і action) та повертає наступний стан (нову версію попереднього).

(previousState, action) => newState

2.7. Redux. Reducers. Чого не можна робити.

Дуже важливо, щоб редюсери залишалися чистими функціями. Ось список того, чого ніколи не можна робити у редюсері:

- Безпосередньо змінювати те, що прийшло у аргументах функції;
- Виконувати будь-які сайд-ефекти: звертатися до API або здійснювати перехід по роутам;
- Викликати не чисті функції, наприклад, `Date.now()` або `Math.random()`.

Лише обчислення нової версії стану!

2.7. Redux. Reducers. Приклад.

Створимо у папці reducers файл **students.js**, і додамо до нього наступний код:

```
const initialState = {
  isLoading: false,
  list: [],
  name: "This is Students!!!!!!",
};

export default (state = initialState, action) => {
  switch (action.type) {
    case 'REQUEST_STUDENTS': {
      return {
        ...state,
        isLoading: true,
      };
    }
    case 'RECEIVE_STUDENTS': {
      const {
        students,
      } = action;
      return {
        ...state,
        isLoading: false,
        list: students,
      };
    }
    default: return state;
  }
};
```


2.7. Redux. Reducers. Підключення до store.

Ми вже знаємо, що для оповіщення store про те, що відбулася якась подія, потрібно передати в неї action, шляхом виклику у нього функції dispatch: **store.dispatch(action)**

Однак, для того, щоб store міг змінити свій state, йому ж потрібно мати цей state!

А state це і є той самий об'єкт, який повертає reducer.

Для підключення редюсера до store, цей редюсер потрібно передати у функцію створення store:

```
store = createStore(reducer);
```

2.7. Redux. Reducers. Декілька редюсерів.

Як тільки додаток стає більш складним, краще розділити функцію-редюсер на окремі функції, які управляють незалежними частинами стану.

Допоміжна функція **combineReducers** перетворює об'єкт, значеннями якого є різні функції редюсери, в одну функцію редюсер, в яку можна передати метод createStore.

Результуючий редюсер викликає вкладені редюсери та збирає їх результати у єдиний об'єкт стану.

Стан, створений іменами combineReducers(), зберігає стан кожного редюсера під їх ключами, переданими в combineReducers()

2.8. Redux. Reducers. combineReducers. Приклад.

```
import { combineReducers } from 'redux';
import studentsReducer from 'reducers/students.js';
import ticketsReducer from 'reducers/tickets.js';

const resultReducer = combineReducers({
  students: studentsReducer,
  tickets: ticketsReducer,
});
const store = createStore(resultReducer);
// Результат;
{
  students: {
    isLoading: false,
    list: [],
    name: "This is Students!!!!",
  },
  tickets: {
    isLoading: false,
    list: [],
    name: "This is Tickets!!!!",
  },
}
```

3. React + Redux.

Для початку варто наголосити, що Redux не має відношення до React. Ви можете створювати Redux-програми за допомогою React, Angular, Ember, jQuery або звичайного JavaScript.

І все-таки, Redux працює особливо добре з такими фреймворками, як React і Dequ, тому що вони дозволяють вам описати UI як функцію стану, і, крім того, Redux вміє змінювати стан (state) додатку у відповідь на екшени (actions), що відбулися.

Для встановлення в проект:

```
npm install --save react-redux
```

3.1. Redux + React. Підключення store.

Приклад підключення нашого store до react-компоненти:

```
import React from 'react';
import { createStore, combineReducers } from 'redux';
import { Provider } from 'react-redux';
import App from './containers/App.jsx';
import studentsReducer from './reducers/students.js';
import ticketsReducer from './reducers/tickets.js';
```

```
const combinedReducer = combineReducers({
  students: studentsReducer,
  tickets: ticketsReducer,
});
```

```
const store = createStore(combinedReducer);
```

```
export default () => (
  <Provider store={store}>
    <App />
  </Provider>
)
```

3.2. Redux + React. Використання в компоненті.

Використання даних із store у компоненті:

```
import React, { Component } from 'react';
import { connect } from 'react-redux';
class App extends Component {
  render() {
    const {
      studentsState,
      ticketsState,
    } = this.props;
    return (
      <div>
        We are starting a loooong, but interesting way...
      </div>
    );
  }
}
const mapReduxStateToProps = reduxState => ({
  studentsState: reduxState.students,
  ticketsState: reduxState.tickets,
});
export default connect(mapReduxStateToProps)(App);
```

3.3. Redux + React. Зміна store з React.

Як ми знаємо, щоб змінити стан стора, необхідно в цього стора викликати функцію **dispatch**, і передати до нього **action** (об'єкт з обов'язковим полем **type**).

Нам потрібно всього 2 речі:

- 1) отримати доступ до функції **dispatch** у нашій компоненті;
- 2) викликати цю функцію, передавши в неї об'єкт з полем **type**, значенням якого буде рядок, що позначає подію яка відбулася (наприклад: **REQUEST_STUDENTS**)

3.4. Redux + React. Зміна store з React.

Щоб отримати доступ до функції **dispatch** у файлі з нашим компонентом необхідно створити функцію **mapDispatchToProps**, яка приймає один аргумент (назвемо його `dispatch`), і повертає його як поле об'єкта:

```
const mapDispatchToProps = dispatch => ({
  dispatch,
});
```

Далі нам потрібно передати цю функцію на виклик функції **connect** другим параметром:

...

```
export default connect(mapReduxStateToProps, mapDispatchToProps)(App);
```

Тепер в **props** нашої компоненти **App** є поле **dispatch**, яке є функцією, яку ми зможемо викликати, передаючи до неї наш **action**

3.5. Redux + React. Зміна store з React.

Тепер реалізуємо функціонал, що після натискання на кнопку ми надсилаємо двох студентів у наш стор.

```
import React, { Component } from 'react';
import { connect } from 'react-redux';

class App extends Component {
  render() {
    return (
      <button
        onClick={() => this.props.dispatch({
          type: 'RECEIVE_STUDENTS',
          students: [{ name: 'Ivan' }, { name: 'Neivan' }]
        })}
      >
        Press me, and I will set students to redux store
      </button>
    );
  }
}

const mapReduxStateToProps = reduxState => ({
  studentsState: reduxState.students,
});

const mapDispatchToProps = dispatch => ({ dispatch });

export default connect(mapReduxStateToProps, mapDispatchToProps)(App);
```

3.6. Redux + React. Зміна store з React.

У хорошій архітектурі всі екшени винесено в окремий файл, а компоненти повинні їх імпортувати:

```
const receiveStudents = students => ({ students, type: 'RECEIVE_STUDENTS' });
const requestStudents = () => ({ type: 'REQUEST_STUDENTS' });
const errorReceiveStudents = () => ({ type: 'ERROR_RECEIVE_STUDENTS' });
const getStudents = (studentsCount) => new Promise((onSuccess) => {
  setTimeout(
    () => onSuccess(Array
      .from(new Array(studentsCount).keys())
      .map(index => ({ name: `Student ${index}` }))),
    1000
  );
});

const fetchStudents = ({ studentsCount }) => (dispatch) => {
  dispatch(requestStudents()); // Повідомляю стору, що роблю запит користувачів
  return getStudents(studentsCount) // Викликаю функцію запиту студентів
    .then(students => dispatch(receiveStudents(students))) // Успіх
    .catch(() => dispatch(errorReceiveStudents())); // Помилка
};

export default {
  fetchStudents,
};
```

3.7. Redux + React. Зміна store з React.

```
import React, { Component } from 'react';
import { connect } from 'react-redux';
import studentsActions from '../actions/students';
class App extends Component {
  render() {
    return (
      <button
        onClick={() => studentsActions.fetchStudents({
          studentsCount: 5,
        })(this.props.dispatch)}
      >
        Press me, and I will set students to redux store
      </button>
    );
  }
}
const mapReduxStateToProps = reduxState => ({
  studentsState: reduxState.students,
});
const mapDispatchToProps = dispatch => ({ dispatch });
export default connect(mapReduxStateToProps, mapDispatchToProps)(App);
```

3.8. Redux + React. Зміна store з React.

На попередньому слайді показано виклик екшен-функції, яка імпортується з файлу з екшенами, але цей виклик трохи незвичний та некрасивий:

```
studentsActions.fetchStudents({  
  studentsCount: 5,  
})(this.props.dispatch)
```

Насправді тут нічого складного, просто функція *fetchStudents* повертає іншу функцію, яка вже приймає **dispatch** своїм аргументом, але факт того, що це некрасиво, залишається фактом...

3.9. Redux + React. bindActionCreators.

bindActionCreators(actionCreators, dispatch) - перетворює об'єкт, значеннями якого є генератори екшенів у об'єкт із тими ж самими ключами, але генераторами екшенів, обгорнутими у виклик **dispatch**, тобто вони можуть бути викликані безпосередньо.

3.11. Redux + React. bindActionCreatorsCreators. Приклад.

```
import React, { Component } from 'react';
import { bindActionCreatorsCreators } from 'redux';
import { connect } from 'react-redux';
import studentsActions from '../actions/students';
class App extends Component {
  render() {
    return (
      <button onClick={() => this.props.actionFetchStudents({ studentsCount: 5})}>
        Press me, and I will set students to redux store
      </button>
    );
  }
}
const mapReduxStateToProps = reduxState => ({ studentsState: reduxState.students });
const mapDispatchToProps = (dispatch) => {
  const {
    fetchStudents,
  } = bindActionCreatorsCreators(studentsActions, dispatch);
  return ({
    actionFetchStudents: fetchStudents,
  });
};
export default connect(mapReduxStateToProps, mapDispatchToProps)(App);
```

3.12. Redux + React. bindActionCreatorsCreators. Проблема.

Тут є одна проблема... Попередній приклад не працюватиме.

Справа в тому, що функція **bindActionCreators** першим аргументом очікує **actionCreators**, який є генератором екшену або об'єктом, значеннями якого є генератори екшенів:

```
bindActionCreators({  
  requestStudents: () => ({ type: 'REQUEST_STUDENTS' }),  
  receiveStudents: () => ({ type: 'RECEIVE_STUDENTS' }),  
}, dispatch)
```

А в нашому випадку ми туди передаємо об'єкт, значеннями якого є функції, які повертають не action (об'єкт із полем type), а іншу функцію...

```
const result = bindActionCreators({  
  fetchStudents: ({ studentsCount }) => (dispatch) => { /* код тіла функції */ },  
}, dispatch);
```

3.12. Redux + React. bindActionCreatorsCreators. Рішення.

Прочитавши помилку, яку пише браузер при натисканні на кнопку (з попереднього прикладу), розуміємо, що нам потрібно встановити якийсь **middleware** і додати його в наш **store**.

Мідлвар (middleware) - це запропонований спосіб розширення Redux за допомогою функцій, що налаштовуються. Мідлвар дозволяє вам обернути метод стора `dispatch` для користі та справи (у нашому випадку дозволяє передавати в **dispatch** не тільки об'єкти (**actions**), а й функції). Ключовою особливістю мідлварів є те, що вони компоновані: кілька мідлварів можна об'єднати разом, де кожен мідлвар не повинен знати, що відбувається до або після нього в ланцюжку.

Одним з таких мідлвар є **redux-thunk**. Давайте його встановимо:

```
npm install redux-thunk --save
```


3.12. Redux + React. bindActionCreatorsCreators. Рішення.

Тепер необхідно внести трохи змін у процес створення нашого стору:

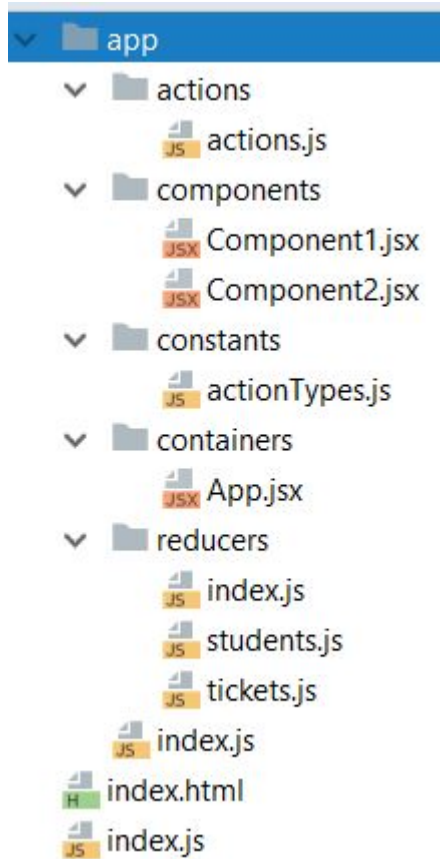
```
import React from 'react';
import { applyMiddleware, createStore, combineReducers } from 'redux';
import { Provider } from 'react-redux';
import { thunk } from 'redux-thunk';
import App from './containers/App.jsx';
import studentsReducer from './reducers/students.js';
import ticketsReducer from './reducers/tickets.js';
const combinedReducer = combineReducers({
  students: studentsReducer,
  tickets: ticketsReducer,
});
const store = createStore(
  combinedReducer,
  applyMiddleware(thunk),
);
export default () => (
  <Provider store={store}>
    <App />
  </Provider>
)
```

4. Презентаційні компоненти та компоненти-контейнери.

React байндинг для Redux відокремлюють презентаційні компоненти від компонент-контейнерів. Такий підхід може полегшити розуміння програми та спростити повторне використання компонентів. Ось короткий виклад різниці між презентаційними та контейнерними компонентами:

	Презентаційні компоненти	Компоненти-контейнери
Призначення	Як виглядає (розмітка, стилі)	Як працює (загрузка даних, оновлення стану)
Знають про Redux	Ні	Так
Читають дані	Читають дані з props	Підписуються на Redux-стан
Змінюють дані	Викликають колбеки з props	Відправляють Redux-екшени

4.1. Приклад структури проекту.



index.js - виконує код підключення зовнішнього react-container'a до DOM-дерева сторінки:

```
ReactDOM.render(<App />, document.getElementById("root"));
```

app/index.js - виконує підключення store до програми (до **<App />**)

app/actions/actions.js - містить actions, які можуть бути застосовані до store.

app/components/*.jsx - містить презентаційні компоненти.

app/containers/*.jsx - містить компоненти-контейнери.

app/constants/actionTypes.js - містить типи екшенів для редюсерів.

app/reducers/index.js - містить код для об'єднання ред'юсерів (combineReducers)